Informe de seguridad de PharmaBackend

Actividad autónoma de la sesión 10

Estudiante Huaccha _____ Fecha de ejecución _____

Lenguaje de Programación II | Docente Reyna Barreto Benjamin David

Esta actividad verifica la autenticación JWT, los permisos y la gestión de cuentas de PharmaBackend. Se implementaron el cambio de contraseña propia, la sustitución de roles y la activación o desactivación sin borrar el historial. La evidencia manual se registra en la matriz y en los anexos de capturas.

C1 Resumen de las pruebas

La matriz contempla 28 casos: 16 base, cuatro propios de grupos distintos y B-01 a B-08. Registra los totales obtenidos después de ejecutar las peticiones y comprobar las evidencias. Un caso con varias peticiones solo pasa si todas cumplen lo esperado.

Grupo	Casos	Ejecutados	Pasa	Falla	Pendientes
Autenticación	5	____	____	____	____
Token	6	____	____	____	____
Permisos	5	____	____	____	____
Usuarios	3	____	____	____	____
CORS	1	____	____	____	____
Contraseñas	4	____	____	____	____
Roles	2	____	____	____	____
Estado	2	____	____	____	____
Total	28	____	____	____	____

Resultados manuales Por registrar con las capturas. La matriz inicial conserva Pendiente para evitar atribuir resultados antes de medirlos.

Verificación automatizada

El 18 de octubre de 2026 se aprobaron 45 pruebas aisladas sin fallos ni errores. Cubren JWT y permisos, cambio de contraseña, gestión de usuarios y reglas HTTP con MockMvc. No utilizaron Oracle ni reemplazan los 28 casos manuales. La prueba de contexto que requiere la base de datos no se ejecutó.

C2 Fichas de hallazgos

Registra una ficha por cada caso que falle. Si todos los casos ejecutados pasan, indica que no se identificaron hallazgos en esa ejecución y conserva sus evidencias. Los casos pendientes no se consideran aprobados.

Ficha para un caso fallido

ID del caso y título _____

Descripción de lo observado _____

_____

Severidad y justificación _____

Evidencia Nombre del archivo y petición _____

Causa probable Clase y método _____

_____

Corrección propuesta _____

_____

Reprueba y resultado _____

Aplicación inmediata de roles y estado

JwtAuthenticationFilter.doFilterInternal obtiene el username del JWT validado y llama a UsuarioDetailsService.loadUserByUsername en cada petición. Las authorities se construyen de nuevo desde los roles actuales y la cuenta solo se autentica si está habilitada. Por eso el mismo token adquiere o pierde permisos después de B-05 y recibe 401 al desactivar la cuenta en B-07. Los claims antiguos del JWT no sustituyen la consulta actual.

Alcance de la entrega

La rama es feature/seguridad-pruebas-huaccha y la solicitud se dirige a main. Los tres commits tienen fecha real del 18 de octubre; el criterio de días distintos queda pendiente. Se conserva la base existente y la decisión de no configurar Flyway en esta entrega.

C3 Análisis de seguridad

Mensajes uniformes al iniciar sesión

GlobalExceptionHandler.handleAuthentication responde 401 con Usuario o contraseña incorrectos ante las excepciones de autenticación. La misma respuesta para usuario inexistente, contraseña incorrecta y cuenta deshabilitada evita confirmar qué cuentas están registradas o cuáles se encuentran activas. Un mensaje distinto permitiría enumerar usuarios y concentrar los intentos de acceso sobre cuentas reales. Los detalles internos quedan fuera de la respuesta; no se registran contraseñas. Esta uniformidad no elimina por sí sola diferencias de tiempo ni sustituye un control de intentos.

Vigencia del token después del cambio de contraseña

AuthServiceImpl.cambiarPassword reemplaza el hash con BCrypt, pero un JWT ya emitido conserva una firma válida y su fecha exp. JwtService.leerClaims comprueba firma y expiración; el filtro revisa estado y permisos, sin comparar el token con la contraseña anterior. Por eso cambiarla no revoca automáticamente el token. Para invalidarlo se podría mantener una versión de sesión por usuario, incluirla en el JWT y compararla en cada petición, incrementándola al cambiar la contraseña. Se necesitaría también una política de revocación para tokens de renovación si se incorporan.

Consecuencias de publicar el secreto JWT

Quien conoce el secreto podría firmar tokens que superen la verificación criptográfica. En esta implementación, suplantar el sub de un usuario existente permitiría usar sus permisos actuales, porque el filtro consulta la cuenta en la base. Actuaría de inmediato rotando el secreto y reiniciando las instancias para invalidar los tokens anteriores, retirando el valor expuesto del código e historial y revisando accesos. En producción se debe obtener de una variable de entorno o un gestor de secretos. Eliminar la publicación sin rotar la clave no corrige la exposición.

Protección de reportes con PreAuthorize

ReporteController declara @PreAuthorize con REPORTES_VER a nivel de clase: mantiene la regla junto a las operaciones y evita depender únicamente de que cada URL esté escrita en SecurityConfig. Permite evaluar permisos y argumentos del método cuando se necesite. Su desventaja es distribuir las reglas entre la cadena de filtros y los controladores; requiere @EnableMethodSecurity y pruebas para evitar omisiones. Las anotaciones se aplican mediante proxies de Spring, por lo que las invocaciones internas al mismo objeto necesitan especial atención.

Referencias Guía práctica S10 y actividad autónoma S10 del curso, apartados B y C. Código del proyecto: AuthServiceImpl.cambiarPassword, UsuarioServiceImpl.asignarRoles y cambiarEstado, JwtAuthenticationFilter.doFilterInternal, JwtService.leerClaims y ReporteController.